In [12]:
```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf

from sklearn.preprocessing import MinMaxScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense

print("Libraries imported successfully!")
print("TensorFlow Version:", tf.__version__)
```

```
Libraries imported successfully!
TensorFlow Version: 2.21.0
```

In [13]:
```
data = pd.read_csv(
    r"C:\Users\lenovo\OneDrive\Documents\Desktop\experiment 12\nifty_fifty\NSE_D
)

print("Dataset loaded successfully!")
print("Original Dataset Shape:", data.shape)
```

```
Dataset loaded successfully!
Original Dataset Shape: (114419, 15)
```

In [14]:
```
print("First five rows:")
display(data.head())

print("\nDataset columns:")
print(data.columns.tolist())

print("\nDataset information:")
data.info()
```

```
First five rows:
```

	DATETIME	SYMBOL	OPEN	HIGH	LOW	CLOSE	VOLUME	PREVIOUS_CLOSE
0	10-02-1997	NIFTYJR	1062.79	1065.45	1060.98	1061.69	0.0	NaN
1	11-02-1997	NIFTYJR	1061.11	1065.47	1061.11	1065.14	0.0	1061.69
2	12-02-1997	NIFTYJR	1067.22	1074.53	1067.22	1071.88	0.0	1065.14
3	13-02-1997	NIFTYJR	1071.22	1073.74	1069.54	1073.74	0.0	1071.88
4	14-02-1997	NIFTYJR	1076.96	1078.16	1075.19	1075.29	0.0	1073.74

```
Dataset columns:
['DATETIME', 'SYMBOL', 'OPEN', 'HIGH', 'LOW', 'CLOSE', 'VOLUME', 'PREVIOUS_CLOSE'
, 'CHANGE', 'CHANGE_PERCENT', 'LOW_CLOSE', 'HIGH_CLOSE', 'TOTAL', 'GAP', 'INTERVA
L']

Dataset information:
<class 'pandas.DataFrame'>
RangeIndex: 114419 entries, 0 to 114418
Data columns (total 15 columns):
 #   Column          Non-Null Count   Dtype
---  ------          --------------   -----
 0   DATETIME        114419 non-null  str
 1   SYMBOL          114419 non-null  str
 2   OPEN            114419 non-null  float64
 3   HIGH            114419 non-null  float64
 4   LOW             114419 non-null  float64
 5   CLOSE           114419 non-null  float64
 6   VOLUME          114419 non-null  float64
 7   PREVIOUS_CLOSE  114418 non-null  float64
 8   CHANGE          114418 non-null  float64
 9   CHANGE_PERCENT  114418 non-null  float64
 10  LOW_CLOSE       114419 non-null  float64
 11  HIGH_CLOSE      114419 non-null  float64
 12  TOTAL           114419 non-null  float64
 13  GAP             114419 non-null  str
 14  INTERVAL        114419 non-null  str
dtypes: float64(11), str(4)
memory usage: 13.1 MB
```

In [15]:
```
# Remove extra spaces from column names
data.columns = data.columns.str.strip()

# Filter only NIFTY-50 records
data["SYMBOL"] = data["SYMBOL"].astype(str).str.strip()

data = data[data["SYMBOL"] == "NIFTY"].copy()

print("NIFTY-50 records selected successfully!")
print("NIFTY-50 Dataset Shape:", data.shape)

display(data.head())
```

```
NIFTY-50 records selected successfully!
NIFTY-50 Dataset Shape: (7000, 15)
```

	DATETIME	SYMBOL	OPEN	HIGH	LOW	CLOSE	VOLUME	PREVIOUS_C
51724	10-02-1997	NIFTY	981.00	986.75	975.15	981.45	45815001.0	227
51725	11-02-1997	NIFTY	980.95	996.05	978.85	992.75	40527999.0	9
51726	12-02-1997	NIFTY	991.65	1008.80	990.90	1001.00	47173001.0	9
51727	13-02-1997	NIFTY	998.45	1005.10	995.35	1002.15	31743001.0	10
51728	14-02-1997	NIFTY	1001.30	1011.85	1001.10	1009.45	33066000.0	10

In [16]:
```
# Select required columns
data = data[
    ["DATETIME", "OPEN", "HIGH", "LOW", "CLOSE"]
].copy()

# Rename columns
data.columns = [
    "Date", "Open", "High", "Low", "Close"
]

# Convert date into datetime format
data["Date"] = pd.to_datetime(
    data["Date"],
    dayfirst=True,
    errors="coerce"
)

# Convert numerical columns
for col in ["Open", "High", "Low", "Close"]:
    data[col] = pd.to_numeric(
        data[col],
        errors="coerce"
    )

# Remove missing values
data = data.dropna()

# Sort records chronologically
data = data.sort_values(
    "Date"
).reset_index(drop=True)

print("Data preprocessing completed!")
print("Cleaned Dataset Shape:", data.shape)

display(data.head())
```

```
Data preprocessing completed!
Cleaned Dataset Shape: (7000, 5)
```

	Date	Open	High	Low	Close
0	1997-02-10	981.00	986.75	975.15	981.45
1	1997-02-11	980.95	996.05	978.85	992.75
2	1997-02-12	991.65	1008.80	990.90	1001.00
3	1997-02-13	998.45	1005.10	995.35	1002.15
4	1997-02-14	1001.30	1011.85	1001.10	1009.45

In [17]:
```
# Create previous day's closing price
data["Previous_Close"] = data["Close"].shift(1)

# Remove first row containing missing previous close
data = data.dropna().reset_index(drop=True)

print("Previous closing price created successfully!")

display(data.head())

print("\nFinal Dataset Shape:", data.shape)
```

```
Previous closing price created successfully!
```

	Date	Open	High	Low	Close	Previous_Close
0	1997-02-11	980.95	996.05	978.85	992.75	981.45
1	1997-02-12	991.65	1008.80	990.90	1001.00	992.75
2	1997-02-13	998.45	1005.10	995.35	1002.15	1001.00
3	1997-02-14	1001.30	1011.85	1001.10	1009.45	1002.15
4	1997-02-17	1017.75	1024.30	1015.10	1020.10	1009.45

```
Final Dataset Shape: (6999, 6)
```

In [18]:
```
# Create previous day's closing price
data["Previous_Close"] = data["Close"].shift(1)

# Remove first row containing missing previous close
data = data.dropna().reset_index(drop=True)

print("Previous closing price created successfully!")

display(data.head())
```

```
print("\nFinal Dataset Shape:", data.shape)
```

```
Previous closing price created successfully!
```

	Date	Open	High	Low	Close	Previous_Close
0	1997-02-12	991.65	1008.80	990.90	1001.00	992.75
1	1997-02-13	998.45	1005.10	995.35	1002.15	1001.00
2	1997-02-14	1001.30	1011.85	1001.10	1009.45	1002.15
3	1997-02-17	1017.75	1024.30	1015.10	1020.10	1009.45
4	1997-02-18	1020.75	1020.85	998.90	1000.75	1020.10

```
Final Dataset Shape: (6998, 6)
```

In [19]:
```
# Select input features
X = data[
    ["Open", "High", "Low", "Previous_Close"]
].values

# Select target variable
y = data[["Close"]].values

print("Input Features Shape:", X.shape)
print("Target Shape:", y.shape)
```

```
Input Features Shape: (6998, 4)
Target Shape: (6998, 1)
```

In [20]:
```
# Calculate split index
split = int(len(data) * 0.8)

# Training data
X_train = X[:split]
y_train = y[:split]

# Testing data
X_test = X[split:]
y_test = y[split:]

print("Training samples:", len(X_train))
print("Testing samples:", len(X_test))

print("Training input shape:", X_train.shape)
print("Testing input shape:", X_test.shape)
```

```
Training samples: 5598
Testing samples: 1400
Training input shape: (5598, 4)
Testing input shape: (1400, 4)
```

In [21]:
```
# Initialize scalers
scaler_X = MinMaxScaler()
scaler_y = MinMaxScaler()

# Scale input features
X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

# Scale target values
y_train_scaled = scaler_y.fit_transform(y_train)

print("Feature scaling completed!")

print("Scaled Training Shape:", X_train_scaled.shape)
print("Scaled Testing Shape:", X_test_scaled.shape)
```

```
Feature scaling completed!
Scaled Training Shape: (5598, 4)
Scaled Testing Shape: (1400, 4)
```

In [22]:
```
# Reshape training data
X_train_scaled = X_train_scaled.reshape(
    X_train_scaled.shape[0], 1, 4
)

# Reshape testing data
X_test_scaled = X_test_scaled.reshape(
    X_test_scaled.shape[0], 1, 4
)

print("Data reshaped for LSTM successfully!")

print("Training LSTM Input Shape:", X_train_scaled.shape)
print("Testing LSTM Input Shape:", X_test_scaled.shape)
```

```
Data reshaped for LSTM successfully!
Training LSTM Input Shape: (5598, 1, 4)
Testing LSTM Input Shape: (1400, 1, 4)
```

In [23]:
```
# Build LSTM model
model = Sequential()

# LSTM Layer with 64 units
model.add(
    LSTM(64, input_shape=(1, 4))
)

# Dense hidden layer
model.add(
    Dense(32, activation="relu")
)

# Output layer
model.add(Dense(1))

# Compile model
model.compile(
    optimizer="adam",
    loss="mean_squared_error"
)

# Display model summary
model.summary()
```

```
WARNING:tensorflow:TensorFlow GPU support is not available on native Windows for
TensorFlow >= 2.11. Even if CUDA/cuDNN are installed, GPU will not be used. Pleas
e use WSL2 or the TensorFlow-DirectML plugin.
```

```
c:\Users\lenovo\OneDrive\Documents\Desktop\experiment 12\.venv\Lib\site-packages
\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `input_shape`/`inpu
t_dim` argument to a layer. When using Sequential models, prefer using an `Input
(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 64)	17,664
dense (Dense)	(None, 32)	2,080
dense_1 (Dense)	(None, 1)	33

Total params: 19,777 (77.25 KB)

Trainable params: 19,777 (77.25 KB)

Non-trainable params: 0 (0.00 B)

In [24]:
```
history = model.fit(
    X_train_scaled,
    y_train_scaled,
    epochs=50,
    batch_size=32,
    verbose=1
)

print("Model training completed successfully!")
```

```
Epoch 1/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 1s 1ms/step - loss: 0.0199
Epoch 2/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 968us/step - loss: 1.6400e-05
Epoch 3/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 952us/step - loss: 1.5176e-05
Epoch 4/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 918us/step - loss: 1.4919e-05
Epoch 5/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 892us/step - loss: 1.4754e-05
Epoch 6/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 877us/step - loss: 1.5012e-05
Epoch 7/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 997us/step - loss: 1.4505e-05
Epoch 8/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 877us/step - loss: 1.4492e-05
Epoch 9/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.5254e-05
Epoch 10/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 924us/step - loss: 1.5733e-05
Epoch 11/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 957us/step - loss: 1.4182e-05
Epoch 12/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 917us/step - loss: 1.4025e-05
Epoch 13/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 987us/step - loss: 1.4609e-05
Epoch 14/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 958us/step - loss: 1.4143e-05
Epoch 15/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 936us/step - loss: 1.3880e-05
Epoch 16/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 942us/step - loss: 1.4045e-05
Epoch 17/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 963us/step - loss: 1.4131e-05
Epoch 18/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.3898e-05
Epoch 19/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 957us/step - loss: 1.5150e-05
Epoch 20/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.3873e-05
Epoch 21/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 966us/step - loss: 1.3921e-05
Epoch 22/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 929us/step - loss: 1.6258e-05
Epoch 23/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 966us/step - loss: 1.5190e-05
Epoch 24/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 917us/step - loss: 1.4927e-05
Epoch 25/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 909us/step - loss: 1.3033e-05
Epoch 26/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 909us/step - loss: 1.4103e-05
Epoch 27/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.3205e-05
Epoch 28/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 909us/step - loss: 1.4127e-05
Epoch 29/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 934us/step - loss: 1.4136e-05
Epoch 30/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 937us/step - loss: 1.2001e-05
```

```
Epoch 31/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 930us/step - loss: 1.7383e-05
Epoch 32/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 958us/step - loss: 1.3723e-05
Epoch 33/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 886us/step - loss: 1.2221e-05
Epoch 34/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 956us/step - loss: 1.5952e-05
Epoch 35/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 878us/step - loss: 1.2388e-05
Epoch 36/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.2354e-05
Epoch 37/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 918us/step - loss: 1.3665e-05
Epoch 38/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 918us/step - loss: 1.4902e-05
Epoch 39/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 939us/step - loss: 1.2026e-05
Epoch 40/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 886us/step - loss: 1.2613e-05
Epoch 41/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.2798e-05
Epoch 42/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 878us/step - loss: 1.4299e-05
Epoch 43/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 919us/step - loss: 1.1584e-05
Epoch 44/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 936us/step - loss: 1.3994e-05
Epoch 45/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.8513e-05
Epoch 46/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 918us/step - loss: 1.0363e-05
Epoch 47/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 935us/step - loss: 1.2700e-05
Epoch 48/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 878us/step - loss: 1.0908e-05
Epoch 49/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 877us/step - loss: 1.1359e-05
Epoch 50/50
175/175 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 1.0074e-05
Model training completed successfully!
```

In [25]:
```python
# Generate predictions
predicted_scaled = model.predict(X_test_scaled)

# Convert predictions to original price scale
predicted_prices = scaler_y.inverse_transform(
    predicted_scaled
)

# Actual closing prices
actual_prices = y_test

print("Prediction completed successfully!")
```

```
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
Prediction completed successfully!
```

In [26]:
```python
results = pd.DataFrame({
    "Actual Closing Price": actual_prices.flatten(),
    "Predicted Closing Price": predicted_prices.flatten()
})

print("Actual vs Predicted Closing Prices:")
display(results.head(10))
```

```
Actual vs Predicted Closing Prices:
```

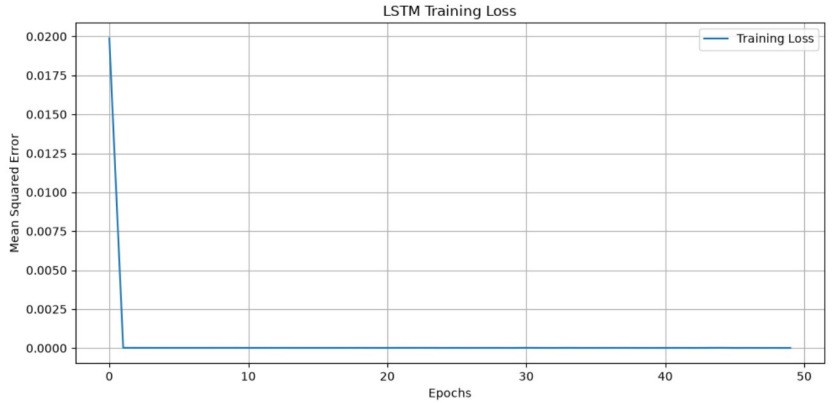
	Actual Closing Price	Predicted Closing Price
0	11440.20	11541.779297
1	11571.20	11578.782227
2	11512.40	11591.863281
3	11474.45	11498.175781
4	11359.90	11458.252930
5	11314.00	11360.927734
6	11174.75	11335.842773
7	11126.40	11222.155273
8	11313.30	11250.534180
9	11234.55	11299.516602

In [27]:
```python
plt.figure(figsize=(10, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.title("LSTM Training Loss")
plt.xlabel("Epochs")
plt.ylabel("Mean Squared Error")

plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



In [28]:
```
plt.figure(figsize=(14, 6))

plt.plot(
    actual_prices,
    label="Actual Closing Price",
    linewidth=2
)

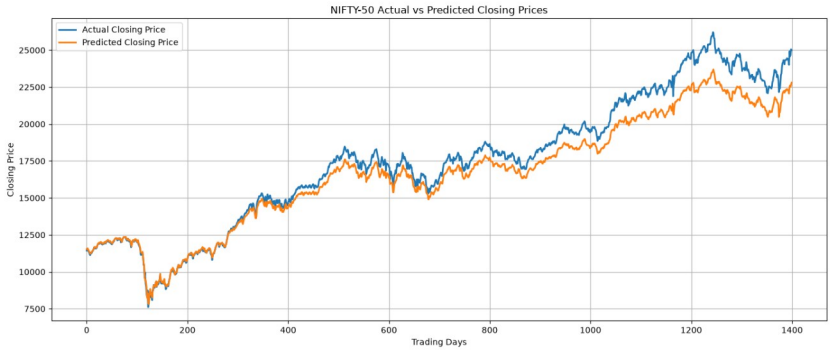
plt.plot(
    predicted_prices,
    label="Predicted Closing Price",
    linewidth=2
)

plt.title("NIFTY-50 Actual vs Predicted Closing Prices")

plt.xlabel("Trading Days")
plt.ylabel("Closing Price")

plt.legend()
plt.grid(True)
plt.tight_layout()

plt.show()
```



In [29]:
```
print("========================================")
print(" NIFTY-50 TIME SERIES FORECASTING")
print("========================================")

print("Original Dataset Records: 114419")
print("NIFTY-50 Records:", 7000)
print("Final Cleaned Records:", len(data))

print("Training Samples:", len(X_train))
print("Testing Samples:", len(X_test))

print("Model Parameters:", model.count_params())

print("\nExperiment Completed Successfully!")
```

```
========================================
 NIFTY-50 TIME SERIES FORECASTING
========================================
Original Dataset Records: 114419
NIFTY-50 Records: 7000
Final Cleaned Records: 6998
Training Samples: 5598
Testing Samples: 1400
Model Parameters: 19777

Experiment Completed Successfully!
```

In [ ]: